Скрипты bash. Ветвления

Теория

Конструкция `if`

Синтаксис:

```
if [ условие ]; then
    # код, если условие истинно
elif [ другое_условие ]; then
    # код, если истинно другое_условие
else
    # код, если все условия ложны
fi
```

Основные моменты:

- Квадратные скобки `[ ]` — это команда `test`, которая проверяет условие.
- Между скобками и условиями **обязательны пробелы**: `[ -f file ]`, а не `[-f file]`.
- Условия можно комбинировать с `&&` (И) и `||` (ИЛИ).

Примеры условий для `if`

Проверка файлов:

```
if [ -f "file.txt" ]; then
    echo "Файл существует."
fi

# Опции:
# -f → файл существует и является обычным файлом.
# -d → существует и является директорией.
# -e → файл/директория существует.
# -s → файл не пустой.
# -r → есть право на чтение.
# -w → есть право на запись.
# -x → есть право на выполнение.
```

Сравнение чисел:

```
if [ "$a" -eq 5 ]; then
    echo "a равно 5."
fi

# Операторы:
# -eq → равно.
# -ne → не равно.
# -gt → больше.
# -lt → меньше.
# -ge → больше или равно.
# -le → меньше или равно.
```

Сравнение строк:

```
if [ "$str1" == "hello" ]; then
    echo "Строки равны."
elif [ "$str1" != "world" ]; then
    echo "Строки не равны."
fi
```

Конструкция `case`

Синтаксис:

```
case переменная in
    шаблон1)
        # код для шаблона1
        ;;
    шаблон2|шаблон3)
        # код для шаблона2 или шаблона3
        ;;
    *)
        # код по умолчанию (если ни один шаблон не подошел)
        ;;
esac
```

Особенности:

- Шаблоны могут содержать символы подстановки (`*` , `?` , `[...]`).
- `;;` означает конец блока для шаблона.
- `*` — обработка всех случаев, не попавших в предыдущие шаблоны.

Примеры использования `case`

Определение типа файла по расширению:

```
filename="image.jpg"
case "$filename" in
    *.txt)
        echo "Текстовый файл."
        ;;
    *.jpg|*.png|*.gif)
        echo "Изображение."
        ;;
    *.sh)
        echo "Скрипт Bash."
        ;;
    *)
        echo "Неизвестный тип."
        ;;
esac
```

Обработка аргументов скрипта:

```
case "$1" in
    "start")
        echo "Запуск сервиса..."
        ;;
    "stop")
        echo "Остановка сервиса..."
        ;;
    "restart")
        echo "Перезапуск сервиса..."
        ;;
    *)
        echo "Используйте: $0 {start|stop|restart}"
        ;;
esac
```

Задания

1. Проверка существования файла

Задача: Напишите скрипт, который принимает путь к файлу как аргумент и проверяет:

- Существует ли файл.

- Если существует, выводит "Файл [имя] найден".
- Если нет — "Файл [имя] не существует".
  Пример:

```
./script.sh /etc/passwd → "Файл /etc/passwd найден."
```

---

2. Определение типа файла

Задача: Создайте скрипт, который проверяет, является ли переданный путь файлом или директорией, и выводит соответствующее сообщение.
Пример:

```
./script.sh /var/log → "Это директория."
```

---

3. Анализ расширения файла (case)

Задача: Напишите скрипт, который принимает имя файла и определяет его тип по расширению:

- `.txt` → "Текстовый файл".
- `.jpg`, `.png` → "Изображение".
- `.sh` → "Скрипт Bash".
- Другое → "Неизвестный тип".
  Пример:

```
./script.sh image.jpg → "Изображение."
```

---

4. Поиск слова в файле

Задача: Создайте скрипт, который принимает слово и путь к файлу, ищет слово в файле.

- Если найдено, выводит "Слово [слово] найдено [N] раз".
- Если нет → "Совпадений нет".
  Пример:

```
./script.sh "error" /var/log/syslog → "Слово error найдено 5 раз."
```

Подсказка: Используйте `grep -c` для подсчета вхождений.

---

5. Проверка прав доступа к файлу

Задача: Напишите скрипт, который проверяет, есть ли у текущего пользователя права:

- На чтение (`-r`).
- На запись (`-w`).
- На выполнение (`-x`).
  Выводит результат для переданного файла.
  Пример:

```
./script.sh script.sh → "Чтение: Да, Запись: Да, Выполнение: Нет."
```

Подсказка: Используйте `if` с условиями `-r` , `-w` , `-x` .

Сложное задание (самостоятельная работа)

Задача: Напишите скрипт, который анализирует логи веб-сервера (`access.log`):

1. Принимает путь к логу как аргумент.
2. Если файл не существует, выводит ошибку.
3. Создает директорию `report` , если её нет.
4. Формирует три отчета в `report/` :
   - `top_ips.txt` : Топ-5 IP-адресов по количеству запросов.
   - `errors.txt` : Все строки с кодами ответа 4xx и 5xx.
   - `paths.txt` : Самые популярные URL-пути (топ-5).
     Требования:

- Используйте `grep` , `awk` , `sort` , `uniq` .
- Проверяйте аргументы и права доступа.
  Пример:

```
./analyze_log.sh access.log
```